Test cases & results:

169 automated tests: 167 pass, 2 are flagged as known defects, and none failed.

- **White-box (69 tests):** boundary value analysis and control-flow testing of the input validators, the trip and availability services, and the route and traffic matcher.
- **Black-box (80 tests):** the whole application is started and driven only over HTTP, with expected results taken from the project's documents, covering authentication, registration, a role-by-operation permission table, use-case scenarios and error handling.
- **Defect found:** D1, malformed JSON returns 500 Internal Server Error instead of 400 Bad Request Error (but this is a minor issue)

Testing approach

Technique	Type	How it was applied
Boundary value analysis	White-box	Each limit found in the code is tested one step below, exactly on, and one step above.
Control-flow (branch and path) testing	White-box	Each branch of a guard, parser or matcher is driven at least once, including early exits and error paths.
Equivalence partitioning	Black-box	Inputs are grouped into valid and invalid classes, e.g. allowed versus near-miss values of an enumeration.
Decision table	Black-box	Every operation is tried with each of the four roles (coordinator, patient, caregiver, driver).
Use-case scenario testing	Black-box	Each documented flow is run end to end through the HTTP API, with its alternate flows.
Error guessing and robustness	Black-box	Hostile strings, oversized or malformed bodies, wrong content types, database failures.
Injected-bug (mutation) check	White-box	Small bugs are inserted into the source to confirm the tests notice them.

The boundary values in the white-box suites were read from the source, so they confirm the code matches itself. The black-box suite takes its expectations from the documents (`use-case-map.md`, `apis-and-responsive-design.md`), so it can disagree with the code. It did once: it showed that caregiver signup requires personal details, which an early assumption had missed.

Test environment

All tests run with Node's built-in test runner through `tsx`, from `dialysis-transport-app/backend`, using the command `npm test --workspace backend`.

Item	Detail
Language and runner	TypeScript, `node:test`, executed by `tsx --test`
Application under test	NestJS backend (`AppModule` for the black-box suite; individual services and functions for the white-box suites)
Node versions used	22.23 on Linux (all 169 tests); 22.20 on Windows (the 89 tests that existed before the black-box suite was added)
Version the project asks for	Node 24 (`.nvmrc`); not tried
Replaced dependency	Supabase (login and database) is a scripted stand-in, so tests need no credentials or network
Clock	Date-dependent tests freeze the clock, e.g. around the Singapore midnight rollover at 16:00 UTC
Setup	`npm install` from the project root first; without `reflect-metadata` two files cannot load

Node prints an `ExperimentalWarning` about the clock-mocking API; it is harmless.

White-box tests

The 69 white-box tests are in `input-boundaries.test.ts` (22), `service-boundaries.test.ts` (26) and `route-traffic.test.ts` (21). They test the code's rules directly, at the edges of each limit and along each branch.

Boundary value analysis

Rule under test	Boundary	Values tried
Full name length	100 characters, after trimming	99, 100, 101; padded 100
Local phone number	exactly 8 digits	7, 8, 9
International phone	6–14 subscriber digits, 15 digits in total, `+65` needs 8	5, 6, 14, 15; 15 vs 16 total; 7, 8, 9 for `+65`
Birth date	1900-01-01 up to today in Singapore time	1899-12-31, 1900-01-01, today, tomorrow; 15:59:59.999 vs 16:00:00.000 UTC
Pickup address, unit number, notes	300, 20, 1000 characters	limit and limit + 1
Postal code	exactly 6 digits	5, 6, 7 digits; empty
Session time	00:00–23:59, end after start	equal, 1 minute apart, reversed by 1 minute, 24:00, 23:60
Weekday	1–7 (Sunday = 7)	0, 1, 7, 8

Recurring range must contain the weekday	first occurrence on or before the last day	last day included vs one day short
Availability slot index and count	0–335; at most 336 unique	-1, 0, 335, 336; 335, 336, 337 slots; duplicates
Availability version, template version	0–2147483646; 0–2147483647	each edge and one beyond
Availability week start	must be a Monday	Sunday, Monday, Tuesday; 29 Feb in a non-leap year
Trip list page	1–1,000,000	0, 1, 1000000, 1000001
Trip search text	100 characters, no control characters	99, 100, 101; characters 0x1F, 0x20, 0x7E, 0x7F
Trip status and direction filters	at most 8 and 2 members	8 vs 9; 2 vs 3
Trip version	1–2147483647	0, 1, 2147483647, 2147483648
Planning horizon	the fourth Sunday from today	Monday, Wednesday, Saturday, Sunday: last allowed day and the next
Generation start	not before today (Singapore)	yesterday, today
Pickup time	an exact minute, strictly in the future	now, now + 1 ms, 59.999 s earlier, seconds and milliseconds set

Location	latitude ±90, longitude ±180, accuracy 0–10000 m	each edge and just beyond
Road match distance	40 m	39 m, 41 m (both sides of the route)
Incident distance	100 m	99 m, 101 m; 90 m vs 150 m past a segment end
Road link length	5000 m	4900 m, 5100 m
Direction alignment	cosine at least 0.85	link turned 30° (matches), 33° (rejected)
Slow-traffic speed band	band 3 or lower	3, 4

Control-flow tests

Unit	Paths driven
Availability access guard	missing or malformed header → 401; wrong role, inactive account, no account → 403; success
Availability save	template path versus dated-week path, each calling a different database function; template version without a week → rejected
`treatmentInput`	body guard; omitted kind defaults to recurring; unknown kind; keys valid only for the other kind; single-session weekday (Sunday = 7); replacement fields all-or-nothing

`patientInput` and `driverInput`	status allowed or forbidden by the `includeStatus` switch; optional fields absent versus present and trimmed; structural guards run before field checks
Trip access rules	8 operations × 4 roles, plus inactive accounts, checking whether the database was reached
Trip actions	confirm, cancel, report unavailable, start and complete each call their own database function; only start and complete pass an action flag
Database error mapping	11 error codes mapped to 403, 409, 400, 404 or 503 for trips, 12 for availability
`routeTraffic`	empty route, single point, zero-length sections, empty traffic feed, multi-section route, zero distance, aligned versus opposing and perpendicular roads, nearest road wins, distant roads discarded, incidents near, far and beside a grid-cell edge

Black-box tests

The 80 black-box tests in `blackbox-api.test.ts` start the whole application and send real HTTP requests; they check only status codes and response bodies, never internal calls. Only Supabase is replaced, so rules enforced inside PostgreSQL are out of scope.

Group	Tests	Technique	What is checked
BB-01 to BB-07	8	Equivalence classes, decision table	Health check; unknown routes give 404; all 51 protected endpoints refuse anonymous callers with 401; seven classes of malformed `Authorization` header; valid, expired, outage, unverified and non-Google sessions; deactivated and unregistered accounts.
BB-08 to BB-15	10	Equivalence classes, boundaries	Patient, driver and caregiver may self-register; coordinator and invented roles never; privilege fields rejected; display name 1, 100 and 101 characters; personal details mandatory; duplicate account gives 409; sign-in required. Two tests (BB-D1, BB-D2) are the known defect.
BB-16	40	Decision table	40 operations × 4 roles = 160 permission checks, e.g. only coordinators generate, assign and review recovery; only patients and caregivers confirm or cancel a pickup; only drivers start, complete or share location; only patients rate or set a preferred driver; nobody edits an existing schedule directly.
UC-03, UC-03.1	4	Use-case scenario	Patient creates a recurring schedule; wrong weekday, reversed times, past range and foreign patient are refused; coordinator generates trips; past, reversed, beyond-horizon, malformed and stale requests are refused.
UC-02, UC-06	4	Use-case scenario	Coordinator edits only a patient's Status and Notes; vehicle review needs a reason to reject; vehicle submission needs every field; availability rejects

			duplicates, out-of-range slots and ownership fields, and handles stale and booked-time edits.
UC-04, UC-07, UC-09, UC-11, UC-12	7	Use-case scenario, boundaries	Confirm and cancel with stale, missing and foreign trips (409, 404, 403); report unavailable, start, complete; location validity; rating 1–5 only, patient only; preferred driver set and removed; trip list filters and trip id validation.
ERR-01 to ERR-07	7	Error guessing	Database failures map to 403, 409, 404, 400 or 503 and never leak passwords, host names or codes; read endpoints return 503 during an outage; SQL, script and path-traversal strings cause no server error; prototype-pollution, oversized and deeply nested bodies do not crash the server; authentication is checked before the body, authorisation before validation.

Results

All 169 tests were run together; 167 pass, 2 known-defect tests, and 0 fail.

Test file	Type	Tests	Pass	TODO	Fail
`input-boundaries.test.ts`	White-box: boundary and control flow	22	22	0	0
`service-boundaries.test.ts`	White-box: boundary and control flow	26	26	0	0
`route-traffic.test.ts`	White-box: control flow and boundary	21	21	0	0
`blackbox-api.test.ts`	Black-box: HTTP API	80	78	2	0
Existing tests (accounts, cors, drivers, patients, treatments, routing)	Mixed	20	20	0	0
Total		**169**	**167**	**2**	**0**

The first run of the black-box file had 7 failures. Four were wrong assumptions in the tests, which were corrected: three assumed caregiver signup takes no personal details, and one assumed the demo tools answer 401 when switched off (they answer 403). The other three all came from one genuine defect, D1, and are now isolated in two TODO tests.

Appendix: traceability to use cases

Every implemented use case has at least one test except UC-13, which is not implemented yet; UC-05, UC-08 and UC-10 are covered only for access control or for the matcher, as noted.

Use case	Black-box tests	White-box tests
UC-01 Sign-in and registration	BB-03 to BB-07, BB-08 to BB-15	`personalInput`, `patientInput`, `driverInput` boundaries
UC-02 Profiles and vehicles	role matrix; coordinator Status and Notes only; vehicle review and submission	`input-boundaries` (names, phones, birth dates, addresses)
UC-03 Treatment schedules	schedule creation and its alternate flows; no direct edits; change-request roles	`treatmentInput` control flow and boundaries
UC-03 Trip generation	generation, past, reversed, beyond-horizon and stale requests	planning-horizon and start-date boundaries
UC-03.1 Assignment	roles for options, assign and unassign	pickup-time boundaries
UC-04 Operations and history	trip list filters; trip id validation; roles for list and delete	page, search, status, direction, sort boundaries
UC-05 Recovery	only coordinators may generate or approve options	none (recovery logic not directly tested)

UC-06 Availability	duplicates, out-of-range slots, ownership fields, stale and booked-time edits	slot, version, week-start boundaries; save paths
UC-07 Disruption reporting	driver reports unavailable; wrong roles refused	trip action to database-function mapping
UC-08 Routes and traffic	none	`route-traffic.test.ts` (21 tests)
UC-09 Execution and tracking	start, complete, sharing; location validity; driver only	latitude, longitude and accuracy boundaries
UC-10 Notifications	access for all four roles only	none
UC-11, UC-11.1 Pickup actions	confirm and cancel; stale, missing and foreign trips	access decision table; error mapping
UC-12 Feedback and preferences	rating 1–5 patient only; preferred driver set, removed, malformed	none
UC-13 Calendar	not covered (deferred in the project plan)	not covered